미들웨어 동작 원리

1-1. 미들웨어란
1-2. 실행 시점별 훅
1-3. 어디에 쓰는가

1-1. 미들웨어란

> Agent의 입력과 출력 사이에 끼어드는 공통 처리 계층이 왜 필요한지 이해해봅니다.

4일차까지 만든 Agent의 흐름은 단순합니다. 사용자 질문이 들어오면 LLM이 도구 호출을 결정하고 도구를 실행한 뒤 결과를 보고 답변합니다. 이 흐름에는 세 가지 문제가 있습니다.

- 모든 도구 호출이 자동입니다. 위험한 작업도 묻지 않고 그대로 실행됩니다.
- 개인정보가 입력에 섞여 있어도 그대로 외부 모델에 전송됩니다.
- 로깅이나 검증 같은 공통 로직을 넣으려면 모든 노드에 같은 코드를 중복해서 써야 합니다.

이런 공통 관심사를 노드마다 박아 넣지 않고 한 곳에서 처리하는 패턴이 미들웨어입니다. Java 웹 개발의 Servlet Filter나 Spring Interceptor와 정확히 같은 발상입니다. 요청이 핸들러에 닿기 전과 응답이 나가기 전에 바깥에서 공통 로직을 끼워 넣습니다.

노드 코드는 그대로 두고 호출 앞뒤에만 로직을 끼워 넣습니다. 로깅 코드가 Agent 구현 안에 박혀 있으면 고칠 때마다 찾아다녀야 합니다. 그런데 미들웨어로 분리하면 등록 리스트에서 한 줄만 빼도 켜고 끌 수 있습니다.

노드 안에 print를 넣어도 되지 않느냐는 의문이 들 수 있습니다. 됩니다. 그러나 노드가 10개면 10곳을 고쳐야 하고 끄고 싶을 때도 10곳을 지워야 합니다. 이것이 관심사 분리의 이득입니다.

> 공통 관심사는 노드가 아니라 미들웨어 계층에서 처리한다는 원칙을 이해했습니다.

1-2. 실행 시점별 훅

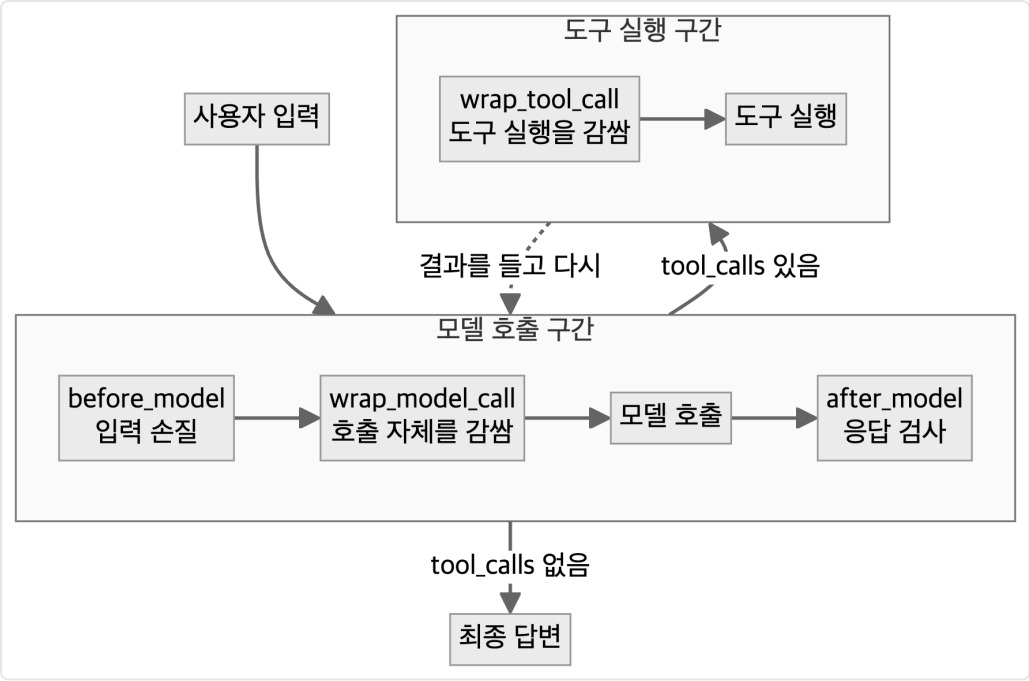
> 모델 호출 전후와 도구 호출 전후에 개입하는 네 가지 훅의 위치를 눈으로 확인해봅니다.

미들웨어는 정해진 시점에 호출되는 메서드(훅, hook)를 구현하는 방식으로 만듭니다. 훅은 정해진 순간에 프레임워크가 대신 불러 주는 함수입니다. `AgentMiddleware`를 상속하고 필요한 훅만 오버라이드하면 됩니다.

파이썬 클래스가 오늘 처음 나오므로 문법만 짚고 갑니다. `class HookTraceMiddleware(AgentMiddleware):` 처럼 **괄호 안에 부모 클래스를 적는 것이 상속**입니다. Java의 `extends` 에 해당합니다. 메서드는 부모와 같은 이름으로 다시 정의하면 그대로 오버라이드되며 `@Override` 같은 표시는 없습니다(그래서 메서드 이름을 오타 내면 조용히 안 불립니다. 훅이 안 먹힌다면 이름부터 확인하세요). 메서드의 첫 인자 `self` 는 Java의 `this` 와 같은 것인데, 파이썬은 생략하지 않고 명시적으로 적어 줍니다.

훅	실행 시점	대표 용도	주의점
`before_model(state, runtime)`	모델 호출 직전	시스템 메시지 주입, 입력 마스킹, 로깅	State 변경만 가능하고 모델 호출 자체를 막을 수는 없음
`after_model(state, runtime)`	모델 호출 직후	응답 검증, 출력 마스킹, 토큰 집계	도구 실행 전에 응답(tool_calls 포함)을 검사할 수 있음
`wrap_model_call(request, handler)`	모델 호출을 감싸는 시점	호출 차단, 재시도, 모델 교체	`handler(request)` 를 부르지 않으면 모델 호출 자체가 생략됨
`wrap_tool_call(request, handler)`	도구 호출을 감싸는 시점	도구 실행 차단, 인자 검사, 실행 시간 측정	`handler(request)` 를 부르지 않으면 도구가 실행되지 않음

네 훅이 Agent 루프의 어디에 위치하는지 보면 다음과 같습니다.



훅의 반환값 규칙은 LangGraph 노드와 같습니다. `None` 을 반환하면 State 변경 없이 그대로 통과하고 dict를 반환하면 State의 일부를 갱신합니다.

네 훅 중에서도 `before_model` 과 `wrap_model_call` 의 차이가 특히 중요합니다. `before_model` 은 모델에 들어갈 내용을 고치는 훅이고 `wrap_model_call` 은 모델 호출 자체를 할지 말지 결정하는 훅입니다.

프롬프트 주입을 발견해 모델을 아예 부르지 않고 거절 메시지를 돌려주는 경우를 생각해 봅니다. 이때는 `wrap_model_call` 에서 `handler` 를 부르지 않고 `AIMessage` 를 직접 반환해야 합니다. 이 구분은 4번째 세션 입력 가드레일에서 다시 씁니다.

한 가지 바뀌는 점이 있습니다. 오늘부터 Agent를 만들 때 3일차와 4일차에서 쓰던 `langgraph.prebuilt` 의 `create_react_agent` 대신 `langchain.agents` 의 `create_agent` 를 씁니다.
미들웨어를 끼워 넣을 수 있는 쪽이 이 API이기 때문입니다. 도구를 넘기는 방식과 그래프가 도는 방식은 같고, 프롬프트 인자 이름만 `prompt` 에서 `system_prompt` 로 바뀝니다.

따라하기

1. 실습 폴더를 만듭니다. 가상환경은 1일차에 레포 루트에 만든 것을 그대로 활성화합니다.

```
mkdir day5_practice
cd day5_practice
..\.venv\Scripts\activate
# 오늘 쓰는 langgraph-checkpoint-sqlite는 1일차 requirements.txt에 들어 있습니다 (추가 설치 없음)
# .env는 레포 루트의 것을 그대로 씁니다 (복사 불필요)
```

2. 훅이 실제로 언제 불리는지 확인하는 최소 예제를 `day5_practice/hook_trace.py` 로 저장해 실행합니다.

```python
# hook_trace.py - 네 가지 훅의 호출 순서 관찰
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import AgentMiddleware
from langchain_aws import ChatBedrockConverse
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage

load_dotenv()


@tool
def search_employee(name: str) -> str:
    """삼성SDS 임직원 디렉터리에서 이름으로 직원 정보를 조회한다."""
    # 교육용 더미 데이터입니다
    fake_db = {
        "김하늘": "김하늘 / 클라우드운영팀 / haneul.kim@samsungsds.example.com",
        "박도윤": "박도윤 / 물류플랫폼팀 / doyun.park@samsungsds.example.com",
    }
    return fake_db.get(name, f"{name}을(를) 찾을 수 없습니다.")


class HookTraceMiddleware(AgentMiddleware):
    """모든 훅에 print를 넣어 호출 순서를 관찰하는 미들웨어"""

    def before_model(self, state, runtime):
        print("[1] before_model: 모델 호출 직전")
        return None

    def after_model(self, state, runtime):
        print("[2] after_model: 모델 호출 직후")
        return None

    def wrap_tool_call(self, request, handler):
        print("[3] wrap_tool_call: 도구 실행 직전")
        result = handler(request)
        print("[4] wrap_tool_call: 도구 실행 직후")
        return result


llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)

agent = create_agent(
    model=llm,
```

```
    tools=[search_employee],
    middleware=[HookTraceMiddleware()],
)

def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


result = agent.invoke({"messages": [HumanMessage("김하늘 정보 조회해줘")]})
print("최종 응답:", get_text(result["messages"][-1]))
```

코드에 들어 있는 `get_text` 는 응답 출력용 헬퍼입니다. `ChatBedrockConverse` 는 메시지의 `content` 를 문자열이 아니라 블록 리스트로 돌려주기도 합니다. 그대로 출력하거나 문자열처럼 다루면 이상한 값이 찍히거나 오류가 납니다. 그래서 오늘 코드는 모두 이 헬퍼를 거칩니다.

3. 질문을 "그냥 안녕이라고만 인사해줘"로 바꿔 다시 실행합니다. 도구 호출이 없으면 `before_model` 과 `after_model` 이 한 번씩만 찍히는 것을 확인합니다.

   실행 결과 예시

```
[1] before_model: 모델 호출 직전
[2] after_model: 모델 호출 직후      <- 여기서 모델이 search_employee 호출을 결정
[3] wrap_tool_call: 도구 실행 직전
[4] wrap_tool_call: 도구 실행 직후
[1] before_model: 모델 호출 직전     <- 도구 결과를 들고 모델을 다시 호출
[2] after_model: 모델 호출 직후
최종 응답: 김하늘 님은 클라우드운영팀 소속이고 이메일은 haneul.kim@samsungsds.example.com
입니다.
```

막힐 때

- `before_model` 이 두 번 찍혀서 당황스럽습니다.

  정상입니다. 도구를 쓰는 Agent는 모델, 도구, 모델 순서로 돌기 때문에 모델 훅이 두 번씩 찍힙니다.

- `AgentMiddleware` 가 import되지 않습니다.

  `langchain` 패키지 버전이 낮은 것입니다. `pip install -U langchain langgraph` 로 올립니다.

- Bedrock 자격 증명 오류가 납니다.

  레포 루트에 `.env`가 그대로 있는지 확인합니다.

확인: 도구를 쓰는 질문에서 훅 로그가 모델, 도구, 모델 순서로 두 바퀴 찍히고 도구가 필요 없는 질문에서는 한 바퀴만 찍히면 성공입니다.

> before_model은 모델에 들어갈 내용을 고치고 wrap_model_call은 모델 호출 자체를 막는다는 구분을 익혔습니다.

1-3. 어디에 쓰는가

> 로깅, 요약, 검열, 승인 요청이 각각 어느 훅에 자리 잡는지 정리해봅니다.

무엇을 미들웨어로 만드는지 대표 사례는 다음과 같습니다.

미들웨어	하는 일	주로 쓰는 훅	오늘 어디서
로깅, 추적	모든 요청과 응답을 기록으로 남김	`before_model`, `after_model`	2-1에서 직접 구현
요약	대화가 길어지면 오래된 메시지를 자동 요약	내장 `SummarizationMiddleware`	2-2에서 코드 리딩
마스킹	개인정보를 모델에 보내기 전에 가림	`before_model`	2-2에서 코드 리딩
승인 요청 (HITL)	위험 도구 호출 전에 사람 승인 대기	내장 `HumanInTheLoopMiddleware`	3번째 세션에서 원리를 직접 구현
검열 (가드레일)	위험한 입력과 출력을 차단하거나 정제	`wrap_model_call`, `after_model`	4번째, 5번째 세션에서 구현
호출 횟수 제한	도구 무한 호출 루프 방지	내장 `ToolCallLimitMiddleware`	5-2 종합 적용에서 사용

LangChain이 미리 만들어 둔 내장 미들웨어도 있습니다. 오늘 다루는 대표 3종의 생김새만 먼저 봐 둡니다.

```
from langchain.agents.middleware import (
    SummarizationMiddleware,      # 대화가 길어지면 자동 요약
    ToolCallLimitMiddleware,      # 도구 호출 횟수 제한
    HumanInTheLoopMiddleware,     # 위험 도구는 사람 승인 (3번째 세션에서 원리를 직접 구현)
)

SummarizationMiddleware(
    model=llm,                          # 요약에 쓸 모델
    max_tokens_before_summary=4000,   # 4000 토큰을 넘으면 요약 시작
    messages_to_keep=10,                # 최근 10개 메시지는 원본 유지
)
ToolCallLimitMiddleware(thread_limit=10)   # 한 스레드에서 최대 10번
HumanInTheLoopMiddleware(interrupt_on={"request_business_trip": True})
```

설계 원칙 하나를 미리 세워 둡니다. 한 미들웨어는 한 가지 일만 합니다. 로깅 미들웨어 안에 비용 추적을 같이 넣으면 나중에 로깅만 끄고 싶을 때 곤란해집니다.

> 운영 관심사마다 맞는 훅이 정해져 있다는 지도를 익혔습니다.
